JAVA PROGRAMMING PRACTICES

Synchronized Seat Reservation

Basic Information

Week	Week 12
Source file	SeatReservationDemo.java
Concepts	Threads, synchronization, shared data, join().

Objective / Aim

To implement synchronized seat reservation using the concepts listed in the R23 Java Programming Practices syllabus.

System Requirements

JDK (Java Development Kit) and a text editor or Java IDE.

Algorithm / Logic Description

1. Create a shared Reservation object.
2. Use a synchronized reserve method.
3. Start and join three person threads.

UML / Program Structure

```
Reservation
+ synchronized reserve()
          ▲
Person extends Thread
```

Source Code

```
class Reservation
{
    private int availableSeats = 10;
    synchronized void reserve(String person, int requested)
    {
        System.out.println(person + " entered.");
        System.out.println("Available seats: " + availableSeats + " Requested seats: " + requested);
        if (requested <= availableSeats)
        {
            System.out.println("Seat Available. Reserve now :-)");
            availableSeats -= requested;
            System.out.println(requested + " seats reserved.");
        }
        else System.out.println("Requested seats not available :-)");
        System.out.println(person + " leaving.");
        System.out.println("-----------------------------------------------");
    }
}
class Person extends Thread
{
    private Reservation reservation;
    private int seats;
    Person(Reservation r, String name, int seats)
    {
        super(name);
        reservation = r;
        this.seats = seats;
    }
    public void run()
    {
        reservation.reserve(getName(), seats);
    }
}
public class SeatReservationDemo
{
    public static void main(String[] args)
    {
        Reservation r = new Reservation();
        Person p1 = new Person(r, "Person-1", 5);
        Person p2 = new Person(r, "Person-2", 2);
        Person p3 = new Person(r, "Person-3", 4);
        p1.start();
```

Source Code (continued)

```
        p2.start();
        p3.start();
        try
        {
            p1.join();
            p2.join();
            p3.join();
        }
        catch (InterruptedException e)
        {
            System.out.println("Thread interrupted");
            Thread.currentThread().interrupt();
        }
    }
}
```

Compilation and Execution

```
javac SeatReservationDemo.java
java SeatReservationDemo
```

Output Screen

```
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
-----------------------------------------------
Person-3 entered.
Available seats: 5 Requested seats: 4
Seat Available. Reserve now :-)
4 seats reserved.
Person-3 leaving.
-----------------------------------------------
Person-2 entered.
Available seats: 1 Requested seats: 2
Requested seats not available :-)
Person-2 leaving.
-----------------------------------------------
```

Result

The program demonstrates the listed syllabus concepts. The sample output is based on a compiled execution.